

The covtl-pipeline Language Pack

Installing and calibrating six synthesis languages
(English, French, Spanish, German, Italian, Portuguese)

Frédéric Berthommier

Version 1.1.0 — September 2026

Scope. This manual documents the *installable language package* of the covtl-pipeline articulatory synthesizer: what it contains, how to install and activate a language, how the vowel targets are calibrated on the engine (VTL/JD3), and how to extend the pack with a new language. The engine itself (the Berthommier COVTL model, VocalTractLab synthesis, the JD3 speaker) is documented in the main repository manual (`docs/manual.pdf`); the multi-speaker registry is documented in `docs/speakers.pdf` and the expressive-prosody option in `docs/expressive_prosody.pdf`.

Contents

1	Introduction	2
1.1	What the language pack is	2
1.2	Package layout	2
1.3	Prerequisites	3
2	Quick start	3
2.1	One command on Windows	3
2.2	The three commands by hand	3
2.3	Installing the whole pack step by step	4
2.4	Reading the verify report	4
3	How the pipeline handles a language	5
3.1	The LANG SECTION of <code>constants.py</code>	5
3.2	The language profile (<code>utils/setlang.py</code>)	6
3.3	The g2p chain: lexicon, exceptions, rules	6
3.4	Integration in the repository and the JD3 guard	7
4	Vowel geometry: polar targets and why they do not transfer	7
4.1	Polar coordinates in the Maeda plane	7
4.2	The validated lesson: targets do not transfer	7
4.3	Calibrating a language on the engine	8
4.4	Nasal vowels and schwa-free languages	8

5	Adding a new language	9
6	Licensing and provenance	9
7	Troubleshooting	10
7.1	Windows: console encoding (kOR.⓪ symptom)	10
7.2	KeyError '⓪' (schwa-free languages)	11
7.3	LPC instability on front-rounded vowels	11
7.4	Restore a pristine tree	11
7.5	Verify reports far fewer vowels than expected	11

1 Introduction

1.1 What the language pack is

The covtl-pipeline synthesizes speech by driving the VocalTractLab (VTL) articulatory synthesizer with trajectories computed by the Berthommier COVTL model. Originally the pipeline spoke English only (through the CMUdict grapheme-to-phoneme dictionary). The language pack extends it to six languages:

- **en** — English (CMUdict; the original pipeline calibration, covered by the regression baselines);
- **fr** — French (Wikipron lexicon + rules; LPC-calibrated vowel targets, validated in situ);
- **es** — Spanish, Castilian with *distinción* (Wikipron `spa_latn_ca`);
- **de** — German (Wikipron; tense/lax vowel pairs, final devoicing, *ach/ich*-laut);
- **it** — Italian (Wikipron; no phonological schwa — Tuscan ⓪ is mapped to e);
- **pt** — European Portuguese (Wikipron `por_latn_po`; nasal vowels through the engine ~ convention).

The pack is a *source package*: a directory (`lang_pack/`) at the repository root that the command `python setup_lang.py install` copies into the pipeline tree. Nothing is installed into `vtl_binaries/`, and the JD3 speaker is **never** modified.

1.2 Package layout

```
lang_pack/
|-- profiles/           g2p + language + expressive profiles -> vtl_synth/utis/
|-- lexicons/          Wikipron TSV + license notices    -> vtl_synth/data/
|-- lang_blocks/       per-language LANG SECTION fragments (blocks.py)
|-- integration/       patched pipeline.py / utis__init__.py
|-- manual/            this manual + figures
|-- install_lang.bat   one-command installer (Windows)
|-- install_lang.sh    one-command installer (Linux/macOS)
|-- README.md          presentation, expected verify scores
|-- LICENSE.md         per-component licensing (GPL-3.0+ / CC BY-SA 3.0)
'-- ARCHITECTURE.md    origins and data flow of every component
```

1.3 Prerequisites

- **Python** ≥ 3.9 with **numpy** (≥ 1.22), **scipy** (≥ 1.7), **cmudict**, and the VTL bindings **vocaltractlab-cython** ($\geq 0.0.10$); **Pillow** and **imageio-ffmpeg** for the video output.
- The **JD3 speaker** file in `vtl_synth/data/vtl_binaries/` (shipped with the repository — the pack does not ship and never touches it).
- The VTL binaries (VocalTractLab 2.3/2.4) reachable by the Python bindings.
- A `covtl-pipeline` clone with the **vtl_synth** package importable (for a local checkout, `pip install -e .` from the repository root; the Windows installer does it for you).

The pronunciation lexicons (`lexicons/*.tsv`, about 13 MB total) ship with the pack; [Section 5](#) gives the exact download commands if you prefer to fetch them from Wikipron directly.

2 Quick start

2.1 One command on Windows

Run `lang_pack\install_lang.bat` with the language code as argument (default `fr`):

```
lang_pack\install_lang.bat fr
lang_pack\install_lang.bat es
```

The batch locates Python (`py -3` or `python`), installs the Python dependencies and the pipeline if needed, then chains the three pipeline commands:

```
python setup_lang.py install --lang fr
python setup_lang.py verify
python setup_lang.py show
```

At the end you see the active language and the in-situ vowel report. On Linux/macOS, `bash lang_pack/install_lang.sh fr` does the same.

2.2 The three commands by hand

```
python setup_lang.py install --lang fr    # install + activate fr
python setup_lang.py verify                # in-situ LPC proof
python setup_lang.py show                  # current language
```

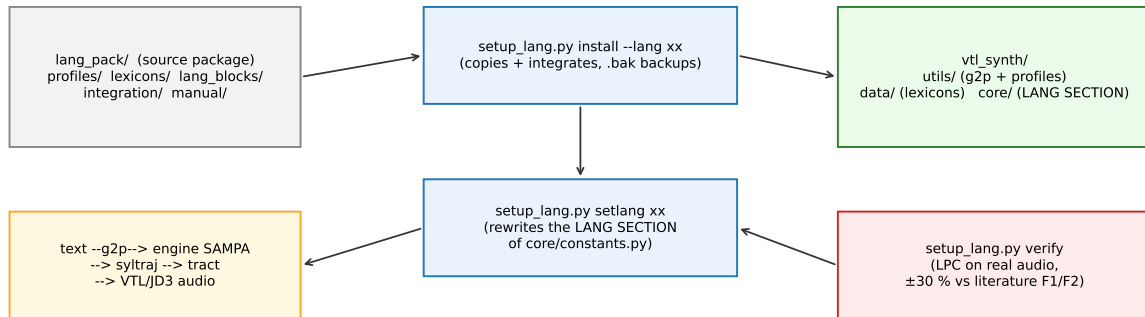
`install` is idempotent: re-running it refreshes the installed modules and rewrites the managed language section of `vtl_synth/core/constants.py`. Every command of `setup_lang.py` prints the currently active language, so a terminal always tells you which language the engine is set to.

Switching language after installation is one command and fully reversible (byte-identical round trip):

```
python setup_lang.py setlang en|fr|es|de|it|pt
python setup_lang.py list                    # available languages
python setup_lang.py restore --purge         # full uninstall
```

2.3 Installing the whole pack step by step

1. Check the prerequisites (Section 1.3).
2. If a lexicon TSV is missing, download it (curl) or copy it from `lang_pack/lexicons/` — see `lexicons/README_xx_lexicon.md` per language. The g2p also works without lexicons (rules + exceptions only), with a warning.
3. `python setup_lang.py install -lang <xx>` — copies the language profile modules into `vtl_synth/utils/`, the lexicons into `vtl_synth/data/`, integrates the language-aware Pipeline (the original files are backed up once as `.bak`), and rewrites the LANG SECTION of `constants.py` for the requested language.
4. `python setup_lang.py setlang <xx>` whenever you want another language.
5. `python setup_lang.py verify` — the proof (below).
6. To roll everything back: `python setup_lang.py restore -purge` restores the pristine pipeline files and removes the installed language modules. Repository-tracked lexicons and user-modified data files are never deleted.



JD3 speaker and vtl_binaries/ are never touched — restore [--purge] returns the tree to its pristine state

Figure 1: Installation and switching data flow. The pack is the *source*; `install` copies/integrates it into the pipeline tree; `setlang` rewrites the managed LANG SECTION; `verify` closes the loop by measuring what the engine actually synthesizes. The JD3 speaker and `vtl_binaries/` are never touched.

2.4 Reading the verify report

`verify` is the in-situ proof that the installed vowel targets produce the intended vowels on *your* engine: for every vowel of the active language it synthesizes the isolated vowel through the real pipeline (SAMPA direct, JD3), measures F1/F2 by LPC on the produced audio, and compares them with the language’s reference formants (literature values for a male speaker of the language; sources listed in Section 6).

voy	F1 mes	F1 ref	F2 mes	F2 ref	verdict
e	300	400	1707	1900	OK

```
o          323          450 |          993          830 | OK
...
```

```
[setup_lang] Bilan : 6/7 voyelles dans la tolerance +-30%
```

A vowel is *OK* when both measured formants are within $\pm 30\%$ of the reference. Compare your *Bilan* line with the expected scores (Table 1): a healthy installation reproduces them (up to ± 1 verdict of LPC jitter). A score far below the published one (e.g. 2/7 for Spanish) indicates a broken installation — typically missing VTL bindings or a corrupted constants section — not a calibration problem.

Table 1: Expected *verify* scores ($\pm 30\%$ tolerance, JD3), of the shipped blocks. *fr*: the “validated” block measures 4/12 under the current chain (see the LPC bias below). *en*: covered by the regression baselines rather than by the LPC chain (its targets predate the verification tool).

language	expected Bilan	notes
en	4/8 (measured)	original calibration; formally covered by the regression baselines
fr	4/12	documented LPC-measurement gap (see the LPC bias below)
es	6/7	/i/ F2 ceiling of the engine (≈ 1300 Hz)
de	10/14	lax + front-rounded vowels: F1 measured low
it	4/7	F1 bias on close vowels; /o/ engine limit
pt	5/8	/o u/ at the tolerance edge (hypotheses)

The documented LPC bias. The linear-prediction analysis chain (11 kHz, pre-emphasis, order 16) systematically measures *F1 too low on close and rounded vowels* (e.g. French /i/: 210 Hz measured vs 280 expected) and is occasionally unstable between runs on front-rounded vowels and the German schwa (F2 of /@/: 1484 then 896 Hz at an identical target). These deviations are *measurement* artifacts, documented per vowel in the comments of the language blocks — they are not installation errors. The per-vowel comments next to each target in *constants.py* carry the measured F1/F2 and the verdict of the calibration run: the proof travels with the constants.

3 How the pipeline handles a language

A language is described at three places in the installed tree.

3.1 The LANG SECTION of *constants.py*

setup_lang.py manages a delimited section of *vtl_synth/core/constants.py*:

```
# === LANG SECTION - BEGIN (managed by setup_lang.py ...)
ACTIVE_LANG: str = 'fr'
VOWEL_TARGETS: Dict[str, Tuple[float, float]] = {
    'i': (0.340, 5 * np.pi / 3), # 300 deg - F2=2200 (parfait)
    ...
}
VOWEL_EFFORT_GAIN: Dict[str, float] = { 'i': 1.5, ... }
# === LANG SECTION - END ===...
```

VOWEL_TARGETS maps each vowel key to its target in the polar (Maeda) plane: radius ρ and angle θ (Chapter 4). *VOWEL_EFFORT_GAIN* compensates the intrinsic intensity gap of high vowels

(clamped to 1.5 downstream). The comments beside each target carry the measured F1/F2 and verdict of the calibration — any target that was never measured in situ is flagged `HYPOTHÈSE` in the block.

3.2 The language profile (`utils/setlang.py`)

Each language registers a `LangProfile` bundling:

- the **g2p gateway** (see below);
- an **F0 declination contour**: `onset_gain` / `final_gain` multipliers of the base F0 (102.216 Hz for JD3), grounded in the prosody literature — en: ToBI guidelines (Beckman & Ayers 1997), fr: Jun & Fougeron (2002), es: Face (2003), de: Grice et al. (2005), it: Avesani (1995), pt: Frota (2000);
- **default syllabic timings** (Table 2).

Table 2: Default transition durations per language (milliseconds). **These are rhythm-type starting hypotheses, not measurements**: they encode the expected syllable-timed vs stress-timed contrast and were tuned by ear on the demo phrases, not derived from corpus statistics.

language	rhythm type	T_{cons} (ms)	T_{voy} (ms)
en	stress-timed	100	80
fr	syllable-timed	140	120
es	syllable-timed	120	100
de	stress-timed	100	90
it	syllable-timed	120	100
pt	syllable-timed	110	100

3.3 The g2p chain: lexicon, exceptions, rules

Every non-English g2p is a three-stage cascade:

1. **Lexicon (absolute priority)**: the Wikipron TSV (`v1l_synth/data/xx_lexicon.tsv`, overridable through the environment variable `$COVTL_XX_LEXICON`). The shared loader (`utils/lexicon_loader.py`) auto-detects engine-key vs IPA entries, converts IPA through the language table, and applies a variant policy (`short` for fr, `grapheme` for es/de/it/pt, with per-language overrides).
2. **Exceptions**: a small built-in dictionary for the frequent irregular words the lexicon misses.
3. **Contextual rules**: the grapheme context rules of the language (e.g. Spanish *ñ*, *ll*, *qu+e/i*, *g+e/i*, *silent h*; German *sch*, *ch*, *ei*, *eu*, *ie*, final devoicing; Italian *gn*, *gli*, *sc(e/i)*, double consonants; Portuguese nasalization in *V+m/n+C*, *ão/ões*, final *-am/-em*, *ç*, *nh*, *lh*, final *s/z*, atonic *-o/-e*).

All g2p modules emit the *engine* SAMPA conventions: nasality as a trailing `~` on the oral base (the trajectory engine strips it before target lookup and extends the velum opening instead), a single rhotic R, uvular/palatal fricatives X/C, velar lateral L.

Wikipron pitfalls (learned the hard way). The lexicons come from the **master** branch of CUNY-CL/wikipron (not main); phones are **space-separated** in the TSV (they must be de-spaced before diphthong conversion: $r\ e\ i \rightarrow R\ e\ j$); nasal vowels may arrive decomposed (two-character entries); Wiktionary clitic variants (*del*, *nel*...) produce duplicate headwords that the variant policy must resolve; Tuscan Italian @ (@) does not exist as an engine key for Italian and is mapped to e; Spanish is split `spa_latn_ca` (Castilian, *distinción*) vs `spa_latn_la` (Latin American, seseo) and Portuguese `por_latn_po` (European) vs `por_latn_bz` (Brazilian) — the pack installs the Castilian and European variants.

3.4 Integration in the repository and the JD3 guard

In the v1.1.0 repository the pack ships pre-integrated: the `lang_pack/` tree is part of the repository and `setup_lang.py` sits at the repository root — no patching step is needed, and `install` remains idempotent (it refreshes the modules and rewrites the managed LANG SECTION). The expressive profiles (`chunking.py`, `prosody_f0.py`) are part of the shipped modules; the option they power is documented in `docs/expressive_prosody.pdf`.

Because the per-language vowel targets are calibrated in situ on JD3, every language-management command carries a guard (`_ensure_jd3()`): `install`, `setlang`, `calibrate` and `verify` first check the active speaker and, if it is not jd3, reinstall jd3 (constants + marker + wheel resource) through `install_speaker.py` before performing the operation. It is a transport mechanism, not a refusal: synthesis is never gated, and switching speakers afterwards remains free — the speaker swap *carries the active LANG SECTION over* to the new speaker’s constants.

Only **JD3 follows the pack**. The four other speakers of the registry (s1, s2, m01, w02) ship and are validated with their *native* vowel targets; to work in that native regime, keep the default language (`en`) and install the speaker with `install_speaker.py` (`docs/speakers.pdf`). Users who alternate languages and speakers should install the language first, then the speaker (workflow A of `docs/expressive_prosody.pdf`).

4 Vowel geometry: polar targets and why they do not transfer

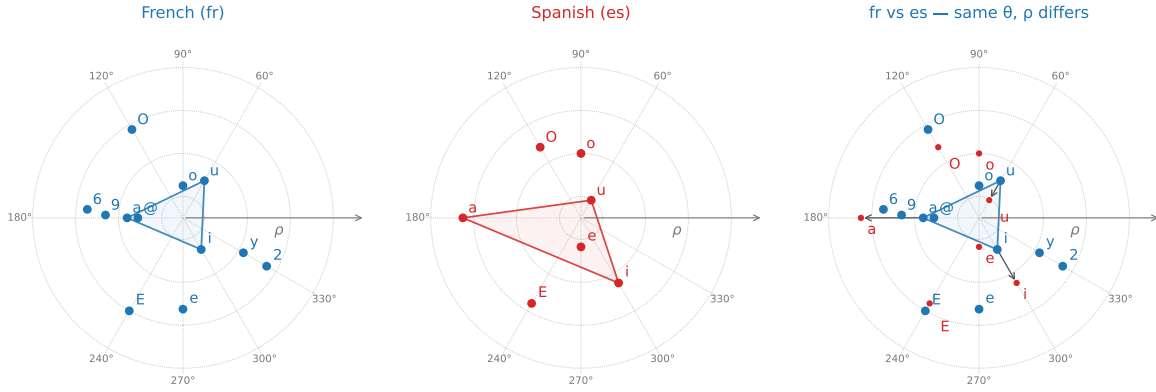
4.1 Polar coordinates in the Maeda plane

The engine places every vowel by two numbers, radius ρ and angle θ , in a polar articulatory plane derived from Maeda’s parametrization. The cardinal angles are stable across languages: *a* at 180°, *u* at 60°, *i* at 300° (and the intermediate vowels on the intermediate spokes: *o* 90°, *Ö* 120°, *E* 240°, *e* 270°, front-rounded *y/2* at 330°). The radius ρ compensates the COVTL coefficient shifts of *this* engine and speaker: it is the calibrated quantity.

4.2 The validated lesson: targets do not transfer

Because ρ absorbs the engine/speaker transfer function *as seen from each language’s reference formants*, a radius calibrated for one language is generally wrong for another, even at the same angle: Figure 2 shows French /i/ and Spanish /i/ sitting at the same spoke but at very different radii, each matching its own language’s F1/F2 reference. Copying vowel targets between languages was tried and rejected during the original in-situ calibration; the pipeline therefore carries one derivation per language and refuses to guess.

Vowel targets in the Maeda (ρ, θ) plane — calibrated per language on VTL/JD3



The cardinal angles θ are shared (a 180°, u 60°, i 300°); the radii ρ are NOT transferable between languages.

Figure 2: Vowel targets of French (blue) and Spanish (red) in the (ρ, θ) plane, read directly from the shipped language blocks. Left/middle: each language with its a-i-u triangle. Right: overlay — the cardinal *angles* are shared, the *radii* differ (French /i/ $\rho = 0.34$ vs Spanish /i/ $\rho = 0.70$ at the same 300° spoke).

4.3 Calibrating a language on the engine

Two commands derive ρ mechanically (never by hand):

1. `python setup_lang.py calibrate <lang>` runs `find_best_rho` for every vowel: it sweeps ρ on the *static* VTL transfer function (tract frozen at the target) and picks the radius whose LPC formants best match the language's reference F1/F2. This is a *starting point* — the gestural dynamics of real synthesis displace the formants.
2. `python setup_lang.py calibrate <lang> -in-situ` then refines each radius through **real synthesis + LPC measurement**: for every vowel it sweeps ρ , applies the candidate to the live `VOWEL_TARGETS`, synthesizes the isolated vowel through the pipeline (JD3) and keeps the radius whose *measured* F1/F2 pass best. The selection criterion is deliberately the same as `verify`'s: first minimize the number of formants outside $\pm 30\%$, then the maximum relative error (not the absolute Hz error, which over-weights F2 and accepts F1 collapses).

The measured F1/F2 and the verdict are written as comments next to each target — the mechanical proof travels with the constants. Anything not measured in situ is flagged `HYPOTHÈSE`.

4.4 Nasal vowels and schwa-free languages

Nasal vowels are not separate targets: the g2p emits the oral base plus a trailing `~` (French *bon* `b0~`, Portuguese *pão* `p@ w`); the syllabic trajectory engine strips the tilde before the target lookup and extends the velum opening (VO) over the vowel instead.

Languages without a phonological schwa (Italian, Spanish) simply have no `@` key in their `VOWEL_TARGETS`. The engine handles this: the closing vowel-vowel arcs of `syltraj` fall back to the neutral anchor (which *is* the schwa by definition) instead of raising `KeyError '@'` — a fix applied once in the engine core (`syltraj._vv_waypoint`) and required by every schwa-free language.

5 Adding a new language

The six built-in languages were added with exactly this recipe; `tests/test_multilang.py` is the executable checklist.

1. **Download the Wikipron lexicon** (master branch; pick the right dialect variant — see the pitfalls in Section 3):

```
curl -L -o vtl_synth/data/xx_lexicon.tsv \  
https://raw.githubusercontent.com/CUNY-CL/wikipron/master/\  
data/scrape/tsv/<xxx_latn_YY>_broad_filtered.tsv
```

Beware: saving the GitHub *page* instead of the raw file gives HTML; a file much smaller than the expected size is suspect. Write the license notice (`README_xx_lexicon.md`) beside it: Wikipron data are Wiktionary extractions under CC BY-SA 3.0.

2. **Write `profiles/g2p_xx.py`** on the three-stage model (lexicon → exceptions → rules) using the shared `lexicon_loader` (`LexiconSpec`: filename, environment variable, IPA→keys table, variant policy).
3. **Declare the vowel inventory** in `profiles/language_vowel_sets.py`: one entry per vowel with the reference F1/F2 for a male speaker of the language (literature — e.g. Quilis 1981 for es, Jørgensen 1969 for de, Ferrero et al. 1978 for it, Delgado-Martins 1999 / Escudero et al. 2009 for pt), the cardinal angle, and a description; give the cardinal corners (*a*, *i*, *u*) and the schwa key (`None` if the language has none).
4. **Register a `LangProfile`** in `profiles/setlang.py`: `g2p` callable, F0 contour with its literature reference, default timings (state that they are hypotheses), notation aliases for the language’s graphemes.
5. **Install and calibrate:**

```
python setup_lang.py install --lang xx  
python setup_lang.py calibrate xx --in-situ  
python setup_lang.py verify
```

Then freeze the calibrated block as the language’s fragment in `lang_blocks/blocks.py` (measured F1/F2 in comments; HYPOTHÈSE flags where the LPC chain could not measure). Adjust `VOWEL_EFFORT_GAIN` after listening (close vowels typically need 1.3–1.5).

6. **Extend the tests:** deterministic `g2p` batteries (rules path with the lexicon variable pointed at a missing file, and lexicon path with the real TSV), a `VOWEL_SETS` inventory test, and — for the ambitious — a slow in-situ verify test on the model of `test_setlang_switch_and_verify_es`.

6 Licensing and provenance

The pipeline code (engine, COVTL model, JD3 handling) is GPL-3.0-or-later. The language pack stacks several origins; Table 3 is the authoritative summary (details and data-flow in `lang_pack/ARCHITECTURE.md` and `lang_pack/LICENSE.md`).

Distribution. GPL-3.0-or-later covers the code; the Wikipron TSV data are CC BY-SA 3.0: redistribution must keep the attribution notices and share the data alike (compatible with the

Table 3: Component provenance of the language pack.

component	origin	license / citation
COVTL engine, JD3 speaker, <code>vtl_binaries/</code> English g2p (CMUdict)	covtl-pipeline (vtl-pipeline reference) CMUdict	GPL-3.0-or-later; JD3 and binaries untouched public domain / BSD-like; source credited
g2p fr/es/de/it/pt, lexicon loader, profiles, blocks	written for this repository	GPL-3.0-or-later
Pronunciation <code>xx_lexicon.tsv</code>	lexicons Wikipron (CUNY-CL/wikipron), extracted from the English Wiktionary	CC BY-SA 3.0 (share-alike); keep the <code>README_xx_lexicon.md</code> notices
Reference formants	Fant 1973; Peterson & Barney 1952; Quilis 1981; Jørgensen 1969; Ferrero et al. 1978; Delgado-Martins 1999; Escudero et al. 2009	academic citation
Prosodic contours	Jun & Fougeron 2002; Face 2003; Grice et al. 2005; Avesani 1995; Frota 2000; Beckman & Ayers 1997 (ToBI)	academic citation

GPL for the distribution of the whole). Local research use carries no formality. If the TSV size is a problem for a repository, do not commit them — the curl commands of Section 5 rebuild them, and the g2p degrades gracefully to rules alone.

Provenance boundary. Everything in the language pack originates from the sources listed in the license file and from no other program or dataset; nothing else may be cited as a component, origin or dependency. The only binaries involved are the VTL 2.3/2.4 `vtl_binaries/` and the JD3 speaker — nothing else.

7 Troubleshooting

7.1 Windows: console encoding (kOR.⓪ symptom)

Symptom. On Windows, accented text passed to the synthesizer comes out wrong: *corée* is decoded with the active OEM code page, the UTF-8 bytes are mis-combined (mojibake of the “corÃ(c)ee” kind), and the French g2p then emits engine keys from the corrupted string — the canonical observed output being kOR.⓪ instead of koRe.⓪ in the demo phrase.

Fix — two invariants.

1. `chcp 65001 >nul` must be the first command of any Windows batch that passes text to the pipeline (it switches the console to UTF-8). It is the first line after `@echo off` in `install_lang.bat` and `launch.bat`; keep it there when you edit those files.
2. The `.bat` files themselves must stay encoded **UTF-8 without BOM** (a BOM breaks the first line at execution).

If you see the kOR.⓪-style symptom, one of the two invariants was broken (typically the file was re-saved by an editor that added a BOM or moved the `chcp` line). Reading this subsection is the one-minute diagnosis.

7.2 KeyError '@' (schwa-free languages)

The trajectory engine used to look up the schwa target when routing closing vowel-vowel arcs; Italian and Spanish have no @. The engine now falls back to the neutral anchor (Section 4). If you ever see `KeyError '@'` again, a custom language block is missing the neutral fallback — check that `syltraj._vv_waypoint` still contains it.

7.3 LPC instability on front-rounded vowels

The `verify` LPC chain can return noticeably different F2 values between runs on close front-rounded vowels and the German schwa (the shipped blocks document a 1484 vs 896 Hz bimodal measurement at an identical target). If a verdict flips between two runs of `verify`, re-run once before drawing conclusions; treat borderline vowels (within a few percent of the $\pm 30\%$ edge) as measurement noise, per the comments in the language blocks.

7.4 Restore a pristine tree

```
python setup_lang.py restore          # original files back
python setup_lang.py restore --purge  # + remove pack modules
```

`restore` puts back the backed-up `constants.py`, `pipeline.py` and `utils/__init__.py` (the `.bak` files are kept for later reinstalls); `--purge` additionally deletes the installed language modules (those listed in the module manifest) and the lexicon files the install itself brought in. Repository-tracked lexicons and anything you modified are never deleted. After a purge the tree is byte-identical to a fresh clone, and `install` brings the pack back at any time.

7.5 Verify reports far fewer vowels than expected

Check that the active language is what you think (`python setup_lang.py show` — every command prints it), that the VTL bindings import (`python -c "import vocaltractlab_cython"`), that the JD3 speaker is present in `vtl_synth/data/vtl_binaries/`, and that the LANG SECTION of `constants.py` still carries both markers. A `Bilan` line consistent with Table 1 means the installation is healthy whatever the individual deviations say.